



SEGi
University &
Colleges



College Name	University of Hertfordshire SEGi College <u>Subang Jaya</u>			
Programme Name	BACHELOR OF <u>CYBERSECURITY AND NETWORKS</u>			
Module Name	CYBER SECURITY AND NETWORKS PROJECT	Module Code	6COM2019	
		Semester	MARCH 2026	
Module Leader	MS ANIZAH ABU BAKAR	Assessment Type	PROJECT REPORT CHAPTER 4	
Lecturer Name	MS NUR DIANA MADINAH			
Student's declaration	I hereby certify that this assignment is my own work and where materials have been used from other resources, they have been properly acknowledged. I also understand I will face the possibility of failing the module if the content of this assignment is plagiarized.			
	No.	Name	Student ID	Signature / Initial
	1	TAN JIAJIAN	SCSJ1901781	TAN
Release Date		Submission Due Date	10/6/2026	Marks obtained:
Date Received		Student's work assessed by / date		

Module Leader’s Feedback.

Contents

CHAPTER 4 IMPLEMENTATION	4
4.1 app.py Function Explanation.....	4
4.2 app.js Function Explanation.....	15

CHAPTER 4 IMPLEMENTATION

4.1 app.py Function Explanation

```
def encrypt_phone(phone):  
    return cipher_suite.encrypt(str(phone).encode()).decode()
```

Encrypts the user's phone number before storing it in the database.

```
def decrypt_phone(cipher_text):  
    return cipher_suite.decrypt(cipher_text.encode()).decode()
```

Decrypts the encrypted phone number when the system needs to display it.

```
def hash_id(phone):  
    return hashlib.sha256((str(phone) + "SECRET PEPPER").encode()).hexdigest()
```

Creates a SHA-256 hashed ID from the phone number. This hashed ID is used as the user's main identifier instead of storing the phone directly.

```
def get_db():  
    conn = sqlite3.connect(DATABASE)  
    conn.row_factory = sqlite3.Row  
    return conn
```

Creates and returns a connection to the SQLite database.

```
@app.route('/signup', methods=['POST'])  
def signup():  
    data = request.get_json()  
    raw_phone = data.get('phone')  
    username = data.get('username')  
    password = data.get('password')  
    public_key = data.get('publicKey')  
    if not raw_phone or not username or not password or not public_key:  
        return jsonify({"message": "Missing registration data"}), 400  
  
    h_id = hash_id(raw_phone)  
    db = get_db()  
  
    existing = db.execute('SELECT 1 FROM users WHERE hashed_id = ?', (h_id,)).fetchone()  
  
    if existing:  
        return jsonify({"message": "This phone number is already registered"}), 400  
  
    enc_phone = encrypt_phone(raw_phone)  
    hashed_pw = generate_password_hash(password)  
  
    try:  
        db.execute(  
            'INSERT INTO users (hashed_id, phone_enc, username, password, publicKey)  
VALUES (?, ?, ?, ?, ?)', (h_id, enc_phone, username, hashed_pw, public_key))  
        db.commit()
```

```

socketio.emit('update_user_list', get_user_list_data())

log_to_admin(
    "AUTH", "New User Registered", f'Phone: {raw_phone} | Username: {username}')
return jsonify({
    "message": "Success", "username": username})
except Exception as e:
    print("Signup error:", e)
    return jsonify({"message": "Registration failed. Please try again."}), 500

```

Handles user registration. It checks the input, prevents duplicate phone registration, encrypts the phone number, hashes the password, stores the public key, and saves the new user.

```

@app.route('/login', methods=['POST'])
def login():
    data = request.get_json()

    phone = data.get('phone', "")
    password = data.get('password', "")
    ip = request.remote_addr

    if detect_sql_injection(phone) or detect_sql_injection(password):
        log_attack(
            "SQL Injection Attempt", ip, "/login", f'phone={phone},
password={password}', "Request Rejected")
        return jsonify({"message": "Suspicious SQL input detected"}), 403

    if detect_xss(phone) or detect_xss(password):
        log_attack(
            "Cross-Site Scripting (XSS) Attempt", ip, "/login", f'phone={phone},
password={password}', "Request Rejected")
        return jsonify({"message": "Suspicious XSS input detected"}), 403

    h_id = hash_id(phone)
    user = get_db().execute(
        'SELECT * FROM users WHERE hashed_id = ?',(h_id,)).fetchone()

    if user and check_password_hash(user['password'], password):
        failed_login_attempts.pop(f'{ip}_{phone}', None)

        log_to_admin(
            "AUTH", "User Login", f'Phone: {phone} authenticated.')

        return jsonify({
            "message": "Success", "username": user['username']})

    key = f'{ip}_{phone}'
    failed_login_attempts[key] = failed_login_attempts.get(key, 0) + 1

    if failed_login_attempts[key] >= 3:
        log_attack(

```

```
"Brute Force Login Attempt", ip, "/login", f'Phone: {phone} | Failed Attempts:
{failed_login_attempts[key]}'", "Suspicious repeated login failures logged")
return jsonify({"message": "Check phone number and password"}), 401
```

Handles user login. It checks for SQL injection and XSS input, verifies the password, and logs brute-force attempts after repeated failed logins.

```
@app.route('/list_users')
def list_users(): return jsonify({"users": get_user_list_data()})
```

Returns the list of registered users to the frontend.

```
def get_user_list_data():
    rows = get_db().execute('SELECT phone_enc, username, hashed_id FROM
users').fetchall()
    return [{"phone": decrypt_phone(r['phone_enc']), "username": r['username']} for r in rows]
```

Retrieves all users from the database and decrypts their phone numbers for display.

```
@app.route('/get_key/<phone>')
def get_key(phone):
    h_id = hash_id(phone)
    user = get_db().execute('SELECT publicKey FROM users WHERE hashed_id = ?',
(h_id,)).fetchone()
    if user: return jsonify({"publicKey": user['publicKey']})
    return jsonify({"message": "Error"}), 404
```

Returns a user's public key so that other users can encrypt messages for them.

```
@app.route('/update_public_key', methods=['POST'])
def update_public_key():
    data = request.get_json()
    phone = data.get('phone')
    public_key = data.get('publicKey')

    h_id = hash_id(phone)
    db = get_db()

    db.execute(
        'UPDATE users SET publicKey=? WHERE hashed_id=?',
        (public_key, h_id)
    )
    db.commit()

    return jsonify({"message": "Public key updated"})
```

Updates a user's public key in the database.

```
@app.route('/fetch_messages/<phone>')
def fetch_messages(phone):
    h_id = hash_id(phone)
    db = get_db()
    rows = db.execute('SELECT * FROM messages WHERE recipient_hash = ? OR
sender_hash = ? ORDER BY timestamp ASC', (h_id, h_id)).fetchall()
    output = []
```

```

for r in rows:
    sender_user = db.execute('SELECT phone_enc FROM users WHERE hashed_id = ?',
(r['sender_hash'],)).fetchone()
    if sender_user:
        output.append({"sender": decrypt_phone(sender_user['phone_enc']), "recipient":
phone, "payload": json.loads(r['payload'])})
    return jsonify({"messages": output})

```

Fetches previous encrypted messages involving the selected user.

```

@app.route('/update_profile', methods=['POST'])
def update_profile():
    data = request.get_json()
    old_p, new_p, name = data.get('oldPhone'), data.get('newPhone'), data.get('username')
    old_pw_input, new_pw_input = data.get('oldPassword'), data.get('newPassword')
    db = get_db()
    old_h_id = hash_id(old_p); new_h_id = hash_id(new_p); new_enc_phone =
encrypt_phone(new_p)
    user = db.execute('SELECT * FROM users WHERE hashed_id = ?',
(old_h_id,)).fetchone()
    if not user: return jsonify({"message": "User not found"}), 404
    final_pw = user['password']
    if old_pw_input and new_pw_input:
        if not check_password_hash(user['password'], old_pw_input): return
jsonify({"message": "Current password incorrect"}), 403
        final_pw = generate_password_hash(new_pw_input)
    try:
        db.execute('UPDATE users SET hashed_id=?, phone_enc=?, username=?, password=?
WHERE hashed_id=?', (new_h_id, new_enc_phone, name, final_pw, old_h_id))
        db.execute('UPDATE messages SET sender_hash=? WHERE sender_hash=?',
(new_h_id, old_h_id))
        db.execute('UPDATE messages SET recipient_hash=? WHERE recipient_hash=?',
(new_h_id, old_h_id))

        db.execute('UPDATE blocks SET blocker_hash=? WHERE blocker_hash=?',
(new_h_id, old_h_id))
        db.execute('UPDATE blocks SET blocked_hash=? WHERE blocked_hash=?',
(new_h_id, old_h_id))

        db.commit()
        socketio.emit('user_updated', {"oldPhone": old_p, "newPhone": new_p, "username":
name})
        socketio.emit('update_user_list', get_user_list_data())
        log_to_admin("AUTH", "Profile Updated", f"User {old_p} changed details.")
        return jsonify({"message": "Success"})
    except: return jsonify({"message": "Phone already taken"}), 400

```

Allows users to update their phone number, username, and password. It also updates related message and block records.

```

@app.route('/deactivate_account', methods=['POST'])
def deactivate_account():

```

```

p = request.get_json().get('phone'); h_id = hash_id(p); db = get_db()
db.execute('DELETE FROM users WHERE hashed_id=?', (h_id,))
db.execute('DELETE FROM messages WHERE sender_hash=? OR recipient_hash=?',
(h_id, h_id))
db.commit()
socketio.emit('update_user_list', get_user_list_data())
log_to_admin("AUTH", "Account Deleted", f"User {p} removed from system.")
return jsonify({"message": "Deleted"})

```

Deletes a user account and removes all messages linked to that account.

```

@app.route('/toggle_block', methods=['POST'])
def toggle_block():
    data = request.get_json(); my_h = hash_id(data.get('myPhone')); target_h =
hash_id(data.get('targetPhone')); db = get_db()
    blocked = db.execute('SELECT 1 FROM blocks WHERE blocker_hash=? AND
blocked_hash=?', (my_h, target_h)).fetchone()
    if blocked:
        db.execute('DELETE FROM blocks WHERE blocker_hash=? AND blocked_hash=?',
(my_h, target_h))
        status = "unblocked"
    else:
        db.execute('INSERT INTO blocks VALUES (?, ?)', (my_h, target_h))
        status = "blocked"
    db.commit()
    log_to_admin("AUTH", f"User {status.upper()}", f'{data.get('myPhone')} {status}
{data.get('targetPhone')}')
    return jsonify({"status": status})

```

Blocks or unblocks another user.

```

@app.route('/get_blocks/<phone>')
def get_blocks(phone):
    h_id = hash_id(phone); rows = get_db().execute('SELECT blocked_hash FROM blocks
WHERE blocker_hash=?', (h_id,)).fetchall(); db = get_db(); phones = []
    for r in rows:
        u = db.execute('SELECT phone_enc FROM users WHERE hashed_id=?',
(r['blocked_hash'],)).fetchone()
        if u: phones.append(decrypt_phone(u['phone_enc']))
    return jsonify({"blockedUsers": phones})

```

Returns the list of users blocked by the current user.

```

@app.route('/admin_register', methods=['POST'])
def admin_register():
    data = request.get_json()
    admin_id = data.get('id')
    password = data.get('password')

    if not admin_id or not password:
        return jsonify({"message": "ID and Password are required"}), 400

    hashed_pw = generate_password_hash(password)

```



```

db = get_db()
try:
    db.execute('INSERT INTO admins (admin_id, password) VALUES (?, ?)', (admin_id,
hashed_pw))
    db.commit()
    return jsonify({"message": "Admin created successfully"})
except Exception as e:
    # IntegrityError happens if admin_id already exists
    return jsonify({"message": "Admin ID already exists"}), 400

```

Registers a new admin account with a hashed password.

```

@app.route('/admin_login', methods=['POST'])
def admin_auth():
    data = request.get_json()
    admin_id = data.get('id')
    password = data.get('password')

    db = get_db()
    admin = db.execute('SELECT * FROM admins WHERE admin_id = ?',
(admin_id,)).fetchone()

    # Check if admin exists and verify the hashed password
    if admin and check_password_hash(admin['password'], password):
        # We use a success status code 200
        return jsonify({"status": "success", "message": "Welcome back Commander"}), 200

    return jsonify({"status": "error", "message": "Invalid Admin Account"}), 401

```

Authenticates admin login by checking the admin ID and password.

```

@app.route('/admin_database_security')
def admin_database_security():
    db = get_db()

    users = db.execute("""
        SELECT hashed_id, phone_enc, username, password
        FROM users
    """).fetchall()

    return jsonify({
        "users": [dict(user) for user in users]
    })

```

Returns database security evidence, such as hashed IDs, encrypted phone numbers, usernames, and hashed passwords.

```

@app.route('/forgot_password_send_otp', methods=['POST'])
def send_otp():
    phone = request.get_json().get('phone')
    h_id = hash_id(phone)
    db = get_db()
    user = db.execute('SELECT 1 FROM users WHERE hashed_id=?', (h_id,)).fetchone()

```

```

if not user:
    return jsonify({"message": "Phone not registered."}), 404
otp = str(random.randint(100000, 999999))
otp_storage[h_id] = {"otp": otp, "expires_at": time.time() + 300}
print(f"\n🔑 [OTP] PHONE: {phone} | CODE: {otp} | EXPIRES IN 5 MINUTES\n")
return jsonify({"message": "OTP generated", "otp": otp})

```

Generates a 6-digit OTP for password recovery and stores it temporarily for 5 minutes.

```

@app.route('/forgot_password_verify_otp', methods=['POST'])
def verify_otp():
    data = request.get_json()
    h_id = hash_id(data.get('phone'))
    user_otp = str(data.get('otp'))
    stored = otp_storage.get(h_id)
    if not stored:
        return jsonify({"message": "OTP not found. Please request a new OTP."}), 400
    if time.time() > stored["expires_at"]:
        del otp_storage[h_id]
        return jsonify({"message": "OTP expired. Please request a new OTP."}), 400
    if stored["otp"] == user_otp:
        return jsonify({"message": "Verified"})
    return jsonify({"message": "Invalid OTP"}), 400

```

Checks whether the OTP entered by the user is correct and not expired.

```

@app.route('/forgot_password_reset', methods=['POST'])
def forgot_password_reset():
    data = request.get_json(); h_id = hash_id(data.get('phone'))
    stored = otp_storage.get(h_id)

    if not stored:
        return jsonify({"message": "OTP not found. Please request a new OTP."}), 403

    if time.time() > stored["expires_at"]:
        del otp_storage[h_id]
        return jsonify({"message": "OTP expired. Please request a new OTP."}), 403

    if stored["otp"] != str(data.get('otp')):
        return jsonify({"message": "Invalid OTP"}), 403

    db = get_db()
    hashed_pw = generate_password_hash(data.get('newPassword'))
    db.execute(
        'UPDATE users SET password=? WHERE hashed_id=?',
        (hashed_pw, h_id)
    )
    db.commit()
    del otp_storage[h_id]

    log_to_admin("AUTH", "Password Reset", f"Account recovered for ID: {data.get('phone')}")

```

```
return jsonify({"message": "Password reset successful"})
```

Resets the user's password after OTP verification and stores the new password as a hash.

```
def log_to_admin(event_type, event_name, details, payload=None):
    socketio.emit('admin_log', {
        "type": event_type,
        "event": event_name,
        "details": details,
        "payload": payload
    }, room='admin_room')
```

Sends system activity logs to the admin monitor in real time.

```
def log_attack(attack_type, source, target, payload, action):
    db = get_db()
    db.execute('INSERT INTO attack_logs (attack_type, source, target, payload, action)
VALUES (?, ?, ?, ?, ?)', (attack_type, source, target, payload, action))
    db.commit()
    socketio.emit('attack_log', {
        "attackType": attack_type,
        "source": source,
        "target": target,
        "payload": payload,
        "action": action
    }, room='admin_room')
```

Stores detected attacks in the database and displays them in the admin monitor.

```
def detect_sql_injection(value):
    if not value:
        return False
    value = str(value).lower()
    patterns = [
        " or 1=1",
        "or 1=1",
        "--",
        "drop table",
        "union select",
        "select * from"
    ]
    return any(p in value for p in patterns)
```

Checks user input for common SQL injection patterns.

```
def detect_xss(value):
    if not value:
        return False
    value = str(value).lower()
    patterns = [
        "<script",
        "</script>",
        "javascript:",
        "onerror=",
    ]
```

```

        "onload=",
        "<img"
    ]
    return any(p in value for p in patterns)

```

Checks user input for common XSS attack patterns.

```

@app.route('/admin_attack_logs')
def admin_attack_logs():
    db = get_db()

    logs = db.execute('SELECT attack_type, source, target, payload, action, timestamp FROM
    attack_logs ORDER BY id DESC').fetchall()

    return jsonify({
        "logs": [dict(log) for log in logs]
    })

```

Returns all recorded attack logs to the admin panel.

```

@socketio.on('join')
def on_join(phone):
    old_sid = active_sessions.get(phone)
    if old_sid and old_sid != request.sid:
        socketio.emit('force_logout', {
            "message": "This account has logged in on another device."
        }, room=old_sid)

    active_sessions[phone] = request.sid
    join_room(phone)

```

Connects a user to their real-time chat room and forces logout if the same account logs in on another device.

```

@socketio.on('join_admin')
def on_join_admin():
    join_room('admin_room')
    print(" 🟡 Admin connected to System Monitor")

```

Connects the admin to the admin monitoring room.

```

@socketio.on('chat_message')
def handle_chat_message(data):
    s_phone = str(data.get('sender')); r_phone = str(data.get('recipient'))
    s_hash = hash_id(s_phone); r_hash = hash_id(r_phone); db = get_db()

    # Admin Log & Terminal Proof
    payload = data.get('payload', {})
    msg_id = payload.get("msgId")

    # Replay Attack Detection
    if msg_id:
        existing = db.execute(
            "SELECT 1 FROM messages WHERE payload LIKE ?",

```

```

        (f%"msgId": "{msg_id}"%',)
    ).fetchone()

    if existing:
        log_attack(
            "Replay Attack Detected",
            s_phone,
            "chat_message",
            f"Duplicate msgId: {msg_id}",
            "Duplicate Message Rejected"
        )
        return

    # Bi-directional block check
    b_rec = db.execute('SELECT 1 FROM blocks WHERE blocker_hash=? AND
blocked_hash=?', (r_hash, s_hash)).fetchone()
    b_sen = db.execute('SELECT 1 FROM blocks WHERE blocker_hash=? AND
blocked_hash=?', (s_hash, r_hash)).fetchone()
    if b_rec or b_sen: return

    # Save to Database
    db.execute('INSERT INTO messages (sender_hash, recipient_hash, payload) VALUES (?,
?, ?), (s_hash, r_hash, json.dumps(data['payload'])))
    db.commit()

    log_to_admin("MSG", "Encrypted Relay", {
        "from": s_phone,
        "to": r_phone,
        "mediaType": payload.get("mediaType", "text").upper(),
        "ciphertext": payload.get("data", ""),
        "wrappedKey": payload.get("aesKey", "")
    })

    print("\n" + "="*60)
    print("🔒 [E2EE PROTOCOL] SECURE MESSAGE RELAY")
    print(f" DIRECTION   : {s_phone} >>> {r_phone}")
    print(f" MEDIA TYPE    : {payload.get('mediaType', 'text').upper()}")
    print("-" * 60)
    print(f" CIPHERTEXT    : {payload.get('data', "")[:100]}...")
    print(f" WRAPPED KEY   : {payload.get('aesKey', "")[:100]}...")
    print("="*60 + "\n")

    recipient_sid = active_sessions.get(r_phone)

    if recipient_sid:
        socketio.emit('receive_message', data, room=recipient_sid)

```

Handles encrypted message transmission between users. The function performs replay attack detection using unique message IDs, checks block relationships between users, stores encrypted payloads in the database, records encrypted communication evidence for administration purposes, and forwards the encrypted message to the intended recipient. Since

the server only relays ciphertext and encrypted AES keys, it cannot access the original message content, supporting the End-to-End Encryption (E2EE) architecture.

```
@socketio.on('edit_message')
def handle_edit(data):
    db = get_db(); r_hash = hash_id(data['recipient'])
    db.execute('UPDATE messages SET payload=? WHERE sender_hash=? AND payload
LIKE ?', (json.dumps(data['newPayload']), hash_id(data['sender']), f"%{data['msgId']}%"))
    db.commit()

    recipient_sid = active_sessions.get(data['recipient'])

    if recipient_sid:
        socketio.emit('message_edited', data, room=recipient_sid)
```

Updates an existing encrypted message payload when the sender edits a message.

```
@socketio.on('delete_message')
def handle_delete(data):
    db = get_db()
    msg_id = data['msgId']

    if data.get('mode') == "everyone":
        marker = {"isDeleted": True, "msgId": msg_id}
        db.execute('UPDATE messages SET payload=? WHERE payload LIKE ?',
(json.dumps(marker), f"%msgId": "{msg_id}%"))
        db.commit()
        recipient_sid = active_sessions.get(data['recipient'])
        if recipient_sid:
            socketio.emit('message_deleted', {"msgId": msg_id, "sender": data['sender']},
room=recipient_sid)
        else:
            db.execute('DELETE FROM messages WHERE sender_hash=? AND payload LIKE
?',(hash_id(data['sender']), f"%msgId": "{msg_id}%"))
            db.commit()
```

Handles message deletion. It can either mark the message as deleted for everyone or delete it from the sender's side only.

4.2 app.js Function Explanation

```
function openDB() {
  return new Promise((resolve, reject) => {
    const request = indexedDB.open(dbName, 2);
    request.onupgradeneeded = (e) => {
      let db = e.target.result;
      if (!db.objectStoreNames.contains(storeName)) db.createObjectStore(storeName);
      if (!db.objectStoreNames.contains(remarkStore)) db.createObjectStore(remarkStore);
    };
    request.onsuccess = () => resolve(request.result);
    request.onerror = () => reject(request.error);
  });
}
```

Opens the browser IndexedDB database for storing chat history and remarks.

```
async function saveHistory() {
  if (!myPhone) return;
  const db = await openDB();
  const tx = db.transaction(storeName, "readwrite");
  tx.objectStore(storeName).put(allConversations, `history_${myPhone}`);
}
```

Saves all local chat conversations into IndexedDB.

```
async function loadHistoryFromDB(phone) {
  const db = await openDB();
  return new Promise((resolve) => {
    const tx = db.transaction(storeName, "readonly");
    const req = tx.objectStore(storeName).get(`history_${phone}`);
    req.onsuccess = () => resolve(req.result || {});
    req.onerror = () => resolve({});
  });
}
```

Loads saved chat history for the current user.

```
async function loadRemarks() {
  const db = await openDB();
  return new Promise((resolve) => {
    const tx = db.transaction(remarkStore, "readonly");
    const request = tx.objectStore(remarkStore).get(`remarks_${myPhone}`);
    request.onsuccess = () => {
      allRemarks = request.result || {};
      resolve();
    };
    request.onerror = () => resolve();
  });
}
```

Loads saved contact remarks or aliases from IndexedDB.

```
function getDisplayName(phone, originalUsername) {  
  return allRemarks[phone] || originalUsername || "Unknown User";  
}
```

Shows the remark name if available; otherwise shows the original username.

```
function openContactInfo() {  
  document.getElementById("info-phone").value = selectedTarget;  
  document.getElementById("info-username").value = selectedUsername;  
  document.getElementById("info-remark").value = allRemarks[selectedTarget] || "";  
  
  // Update Block Button UI  
  const blockBtn = document.querySelector("button[onclick='blockUser()']");  
  if (myBlockedUsers.includes(selectedTarget)) {  
    blockBtn.innerText = "Unblock User";  
    blockBtn.className = "btn btn-success w-100"; // Green for unblock  
  } else {  
    blockBtn.innerText = "Block User";  
    blockBtn.className = "btn btn-danger w-100"; // Red for block  
  }  
  
  document.getElementById("modal-contact-info").classList.remove("hidden");  
}
```

Opens the contact information modal and updates the block/unblock button.

```
function closeContactInfo() {  
  document.getElementById("modal-contact-info").classList.add("hidden");  
}
```

Closes the contact information modal.

```
async function saveRemark() {  
  const remark = document.getElementById("info-remark").value.trim();  
  const phone = document.getElementById("info-phone").value;  
  
  if (remark) {  
    const duplicatePhone = Object.keys(allRemarks).find(p =>  
      p !== phone &&  
      allRemarks[p].toLowerCase() === remark.toLowerCase()  
    );  
  
    if (duplicatePhone) {  
      return alert("This remark name is already used. Please use a different name.");  
    }  
  
    allRemarks[phone] = remark;  
  } else {  
    delete allRemarks[phone];  
  }  
  
  const db = await openDB();  
  const tx = db.transaction(remarkStore, "readwrite");
```



```

tx.objectStore(remarkStore).put(allRemarks, `remarks_${myPhone}`);

alert("Remark saved!");
closeContactInfo();
    document.getElementById("chat-target-name").innerText = getDisplayName(phone,
selectedUsername);
    renderUserList();
}

```

Saves or updates a custom remark name for a contact.

```

async function fetchBlockedUsers() {
    const res = await fetch(`/get_blocks/${myPhone}`);
    const d = await res.json();
    myBlockedUsers = d.blockedUsers || [];
}

```

Gets the current user's blocked user list from the server.

```

async function blockUser() {
    const target = selectedTarget;
    const res = await fetch("/toggle_block", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ myPhone: myPhone, targetPhone: target })
    });

    const d = await res.json();

    if (d.status === "blocked") {
        myBlockedUsers.push(target); // Add to local list
        alert("User Blocked.");
    } else {
        myBlockedUsers = myBlockedUsers.filter(p => p !== target); // Remove from local list
        alert("User Unblocked.");
    }

    closeContactInfo();
    renderUserList();
}

```

Blocks or unblocks the selected user.

```

async function saveSession(phone, keys, username) {
    sessionStorage.setItem("chat_activePhone", phone);
    sessionStorage.setItem("chat_myUsername", username);
    if (keys) {
        const pub = await crypto.subtle.exportKey("jwk", keys.publicKey);
        const priv = await crypto.subtle.exportKey("jwk", keys.privateKey);
        localStorage.setItem(`chat_keys_${phone}`, JSON.stringify({ publicKey: pub,
privateKey: priv }));
    }
}

```

```
}
```

Stores the active session and RSA key pair in browser storage.

```
async function checkExistingSession() {
  const phone = sessionStorage.getItem("chat_activePhone");
  const user = sessionStorage.getItem("chat_myUsername");

  if (!phone || !user) {
    showPage("page-login");
    return;
  }

  let keyData = localStorage.getItem(`chat_keys_${phone}`);

  if (!keyData) {
    myKeys = await crypto.subtle.generateKey(
      {
        name: "RSA-OAEP",
        modulusLength: 2048,
        publicExponent: new Uint8Array([1, 0, 1]),
        hash: "SHA-256"
      },
      true,
      ["encrypt", "decrypt"]
    );

    await saveSession(phone, myKeys, user);

    keyData = localStorage.getItem(`chat_keys_${phone}`);
  }

  try {
    const parsed = JSON.parse(keyData);

    myKeys = {
      publicKey: await crypto.subtle.importKey(
        "jwk",
        parsed.publicKey,
        { name: "RSA-OAEP", hash: "SHA-256" },
        true,
        ["encrypt"]
      ),
      privateKey: await crypto.subtle.importKey(
        "jwk",
        parsed.privateKey,
        { name: "RSA-OAEP", hash: "SHA-256" },
        true,
        ["decrypt"]
      )
    };
  }
```

```

myPhone = phone;
myUsername = user;

allConversations = await loadHistoryFromDB(myPhone);
await loadRemarks();

document.getElementById("display-my-username").innerText = myUsername;
showPage("page-chat");

socket.emit("join", myPhone);
loadUsers();
await fetchOfflineMessages();
addKeyboardSupport();
await fetchBlockedUsers();

} catch (e) {
  console.error(e);
  showPage("page-login");
}
}

```

Checks whether the user already has an active session and restores keys, chat history, remarks, and user list.

```
function logout() { sessionStorage.clear(); location.reload(); }
```

Clears the session and reloads the page.

```

function openProfile() {
  document.getElementById("edit-username").value = myUsername;
  document.getElementById("edit-phone").value = myPhone;
  showPage("page-profile");
}

```

Opens the profile editing page and fills in current user details.

```

async function updateProfile() {
  const newName = document.getElementById("edit-username").value.trim();
  const newPhone = document.getElementById("edit-phone").value.trim();

  const oldPass = document.getElementById("edit-old-pass").value;
  const newPass = document.getElementById("edit-new-pass").value;
  const confirmPass = document.getElementById("edit-confirm-pass").value;

  // 1. Basic Validation
  if (!newName || !newPhone) return alert("Username and Phone cannot be empty.");

  // 2. Username Regex (3-20 chars, alphanumeric)
  const userRegex = /^[a-zA-Z0-9_]{3,20}$/;
  if (!userRegex.test(newName)) {
    return alert("Username must be 3-20 characters (letters, numbers, underscores).");
  }
}

```

```

// 3. Phone Regex (10-11 digits)
const phoneRegex = /^(011\d{8}|01[023456789]\d{7})$/;
if (!phoneRegex.test(newPhone)) {
    return alert("Phone number must be 10 or 11 digits.");
}

// 4. Password Validation (Only if user typed something in password boxes)
if (newPass || oldPass || confirmPass) {
    if (!oldPass) return alert("You must enter your Current Password to set a new one.");
    if (newPass !== confirmPass) return alert("New passwords do not match.");

    // Password Strength Regex
    const passRegex = /^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@$!%*?&._-])[A-Za-z\d@$!%*?&._-]{8,}$/;
    if (!passRegex.test(newPass)) {
        return alert("New password is too weak! Must be 8+ chars with Upper, Lower, Number, and Symbol.");
    }
}

// --- ALL VALIDATION PASSED ---
const res = await fetch("/update_profile", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({
        oldPhone: myPhone,
        newPhone: newPhone,
        username: newName,
        oldPassword: oldPass,
        newPassword: newPass
    })
});

const d = await res.json();
if (!res.ok) return alert(d.message || "Update failed.");

// --- MIGRATION LOGIC (Keys & History) ---
if (newPhone !== myPhone) {
    const keys = localStorage.getItem(`chat_keys_${myPhone}`);
    localStorage.setItem(`chat_keys_${newPhone}`, keys);
    localStorage.removeItem(`chat_keys_${myPhone}`);

    const db = await openDB();
    const tx = db.transaction(storeName, "readwrite");
    const store = tx.objectStore(storeName);

    // Move history to the new ID folder
    if (allConversations) {
        store.put(allConversations, `history_${newPhone}`);
    }
}

```

```

        store.delete(`history_${myPhone}`);
    }
}

// Update session storage
sessionStorage.setItem("chat_activePhone", newPhone);
sessionStorage.setItem("chat_myUsername", newName);

alert("Profile updated successfully!");
location.reload();
}

```

Validates and updates username, phone number, and password.

```

async function deactivateAccount() {
    if (!confirm("Delete account and all history forever?")) return;
    await fetch("/deactivate_account", { method: "POST", headers: { "Content-Type":
"application/json" }, body: JSON.stringify({ phone: myPhone }) });
    localStorage.removeItem(`chat_keys_${myPhone}`);
    logout();
}

```

Deletes the user account and removes local encryption keys.

```

function arrayBufferToBase64(buffer) {
    const bytes = new Uint8Array(buffer);
    let binary = "";
    for (let i = 0; i < bytes.byteLength; i += 8000) binary += String.fromCharCode.apply(null,
bytes.subarray(i, i + 8000));
    return btoa(binary);
}

```

Converts binary encrypted data into Base64 format for storage and transmission.

```

async function decryptPayload(payload) {
    try {
        const decAes = await crypto.subtle.decrypt({ name: "RSA-OAEP" },
myKeys.privateKey, Uint8Array.from(atob(payload.aesKey), c => c.charCodeAt(0)));
        const aesObj = await crypto.subtle.importKey("raw", decAes, { name: "AES-GCM" },
false, ["decrypt"]);
        const decCon = await crypto.subtle.decrypt({ name: "AES-GCM", iv:
Uint8Array.from(atob(payload.iv), c => c.charCodeAt(0)) }, aesObj,
Uint8Array.from(atob(payload.data), c => c.charCodeAt(0)));
        return new TextDecoder().decode(decCon);
    } catch (e) { return null; }
}

```

Decrypts received messages using the user's RSA private key and AES-GCM.

```

async function sendEncrypted(mediaData = null, mediaType = "text") {
    const input = document.getElementById("msg-input");
    const content = mediaData || input.value;
    if (!selectedTarget || (!mediaData && (!content || content.trim() === ""))) {alert("Message
cannot be empty."); return; }
}

```

```

    if (isEditing && !mediaData) { performEdit(content); return; }
    if (myBlockedUsers.includes(selectedTarget)) {
        alert('You have blocked this user. You must unblock them before you can send a message.');
```

```

        return; }

    try {
        const res = await fetch(`/get_key/${selectedTarget}`);
        const k = await res.json();
        const startEncrypt = performance.now();
        const pubKey = await crypto.subtle.importKey("spki",
            Uint8Array.from(atob(k.publicKey), c => c.charCodeAt(0)), { name: "RSA-OAEP", hash:
            "SHA-256" }, false, ["encrypt"]);
        const aes = await crypto.subtle.generateKey({ name: "AES-GCM", length: 256 }, true,
            ["encrypt"]);
        const iv = crypto.getRandomValues(new Uint8Array(12));
        const encC = await crypto.subtle.encrypt({ name: "AES-GCM", iv }, aes, new
            TextEncoder().encode(content));
        const encA = await crypto.subtle.encrypt({ name: "RSA-OAEP" }, pubKey, await
            crypto.subtle.exportKey("raw", aes));
        const msgId = Date.now().toString() + Math.random().toString().slice(2, 8);
        const timestamp = new Date().toISOString();
        const payload = { aesKey: arrayBufferToBase64(encA), iv: arrayBufferToBase64(iv),
            data: arrayBufferToBase64(encC), mediaType, msgId, timestamp };
        const endEncrypt = performance.now();
        console.log("Encryption Time:", (endEncrypt - startEncrypt).toFixed(2), "ms");

        socket.emit("chat_message", { sender: myPhone, recipient: selectedTarget, payload });
        if (!allConversations[selectedTarget]) allConversations[selectedTarget] = [];
        allConversations[selectedTarget].push({ text: content, type: "sent", mediaType, msgId,
            timestamp });
        if (!mediaData) input.value = "";
        await saveHistory(); refreshChatWindow();
    } catch (e) { console.error(e); }
}

```

Encrypts messages before transmission using a hybrid encryption approach. AES-GCM is used to encrypt the message content, while RSA-OAEP is used to encrypt the AES session key. The encrypted payload is then transmitted to the server, ensuring that only the intended recipient can decrypt and read the message.

```

async function performEdit(newText) {
    try {
        const res = await fetch(`/get_key/${selectedTarget}`);
        const k = await res.json();
        const pubKey = await crypto.subtle.importKey("spki",
            Uint8Array.from(atob(k.publicKey), c => c.charCodeAt(0)), { name: "RSA-OAEP", hash:
            "SHA-256" }, false, ["encrypt"]);
        const aes = await crypto.subtle.generateKey({ name: "AES-GCM", length: 256 }, true,
            ["encrypt"]);
        const iv = crypto.getRandomValues(new Uint8Array(12));

```

```

        const encC = await crypto.subtle.encrypt({ name: "AES-GCM", iv }, aes, new
TextEncoder().encode(newText));
        const encA = await crypto.subtle.encrypt({ name: "RSA-OAEP" }, pubKey, await
crypto.subtle.exportKey("raw", aes));
        const newPayload = { aesKey: arrayBufferToBase64(encA), iv:
arrayBufferToBase64(iv), data: arrayBufferToBase64(encC), mediaType: "text", msgId:
editMsgId, isEdited: true };
        socket.emit("edit_message", { msgId: editMsgId, newPayload, sender: myPhone,
recipient: selectedTarget });
        const idx = allConversations[selectedTarget].findIndex(m => m.msgId === editMsgId);
        if (idx !== -1) {allConversations[selectedTarget][idx].text =
newText;allConversations[selectedTarget][idx].isEdited = true;}
        isEditing = false; editMsgId = null; document.getElementById("msg-input").value = "";
document.getElementById("send-btn").innerText = "Send";
        await saveHistory(); refreshChatWindow();
    } catch (e) { console.error(e); }
}

```

Encrypts the edited message again and sends the updated encrypted payload to the server.

```

function deleteMessage(msgId) {
    const msgs = allConversations[selectedTarget];
    const idx = msgs.findIndex(m => m.msgId === msgId);
    if (idx === -1) return;

    const message = msgs[idx];
    const isSentByMe = (message.type === "sent");

    if (isSentByMe) {
        if (!confirm("Delete for everyone?")) return;
        // 1. Change to "Deleted" marker locally
        msgs[idx] = { msgId: msgId, isDeleted: true, type: "sent" };
        socket.emit("delete_message", { msgId, sender: myPhone, recipient: selectedTarget,
mode: "everyone" });
    } else {
        if (!confirm("Delete for me?")) return;
        // 2. Simply remove from screen locally
        msgs.splice(idx, 1);
        socket.emit("delete_message", { msgId, sender: myPhone, recipient: selectedTarget,
mode: "me" });
    }

    saveHistory();
    refreshChatWindow();
}

```

Deletes a message either for everyone or only for the current user.

```

async function deleteConversation(phone) {

    if (!confirm("Delete this chat permanently from your chat list?")) {
        return;
    }
}

```

```

    }
    delete allConversations[phone];
    delete unreadCounts[phone];
    closeContactInfo();
    if (selectedTarget === phone) {
        selectedTarget = "";
        selectedUsername = "";
        document.getElementById("chat-target-name").innerText = "";
        document.getElementById("chat-header").classList.add("hidden");
        document.getElementById("input-area").classList.add("hidden");
        document.getElementById("empty-state").classList.remove("hidden");
        document.getElementById("message-pane").innerHTML = "";
    }
    await saveHistory();
    renderUserList();
}

```

Deletes the whole conversation from the local chat list.

```

function startEdit(msgId, oldText) {
    isEditing = true; editMsgId = msgId;
    document.getElementById("msg-input").value = oldText.replace(" (edited)", "");
    document.getElementById("msg-input").focus();
    document.getElementById("send-btn").innerText = "Update";
}

```

Puts the selected message into edit mode.

```

function copyText(text) { navigator.clipboard.writeText(text.replace(" (edited)", ""));
alert("Copied!"); }

```

Copies a text message to the clipboard.

```

async function processIncomingMessage(data) {
    if (String(data.sender) === String(myPhone)) return;
    const payload = typeof data.payload === 'string' ? JSON.parse(data.payload) :
data.payload;

    if (payload.timestamp) {
        const sentTime = new Date(payload.timestamp).getTime();
        const receivedTime = new Date().getTime();
        console.log("Message Delivery Time:", receivedTime - sentTime, "ms");
    }
    if (payload.isDeleted) return;
    const message = await decryptPayload(payload); if (!message) return;
    if (!allConversations[data.sender]) allConversations[data.sender] = [];
    if (!allConversations[data.sender].some(m => m.msgId === payload.msgId)) {
        allConversations[data.sender].push({ text: message, type: "received", mediaType:
payload.mediaType, msgId: payload.msgId, timestamp: payload.timestamp});
        await saveHistory(); if (String(selectedTarget) !== String(data.sender))
unreadCounts[data.sender] = (unreadCounts[data.sender] || 0) + 1;
    }
    if (String(selectedTarget) === String(data.sender)) refreshChatWindow();
}

```



```

renderUserList();

const senderUser =
window.allUsers.find(
  u => String(u.phone) === String(data.sender)
);

const senderName =
  senderUser?.username || data.sender;

if (
  !isLoadingOfflineMessages &&
  "Notification" in window &&
  Notification.permission === "granted"
) {
  const notification = new Notification(senderName, {
    body: "New message received",
    icon: "/static/images/chipchat.png"
  });

  notification.onclick = function () {
    window.focus();
  };
}
}

```

Receives encrypted messages, decrypts them, saves them locally, updates unread counts, and shows notifications.

```

async function fetchOfflineMessages() {
  isLoadingOfflineMessages = true;

  try {
    const res = await fetch(`/fetch_messages/${myPhone}`);
    const d = await res.json();

    if (d.messages) {
      for (let m of d.messages) {
        if (String(m.recipient) === String(myPhone)) {
          await processIncomingMessage(m);
        }
      }
    }
  } finally {
    isLoadingOfflineMessages = false;
  }
}

```

Loads previous messages from the server when the user reconnects.

```

function renderUserList() {
  const l = document.getElementById("user-list");

```

```

if (!l) return;
l.innerHTML = "";

const searchBox = document.getElementById("user-search");
const term = searchBox ? searchBox.value.trim() : "";

let shownCount = 0;

window.allUsers.forEach(u => {
  if (String(u.phone) === String(myPhone)) return;

  const phoneNumber = String(u.phone);
  const displayName = getDisplayName(u.phone, u.username).toLowerCase();
  const searchTerm = term.toLowerCase();

  const hasHistory =
    allConversations[u.phone] &&
    allConversations[u.phone].length > 0;

  let shouldShow = false;

  if (term === "") {
    shouldShow = hasHistory;
  } else {
    if (hasHistory) {
      shouldShow =
        phoneNumber.includes(term) ||
        displayName.includes(searchTerm);
    } else {
      shouldShow = phoneNumber === term;
    }
  }

  if (!shouldShow) return;

  const d = document.createElement("div");
  d.className = "user-item" + (selectedTarget === u.phone ? " active-user" : "");

  d.innerHTML = `<div><strong>${getDisplayName(u.phone,
u.username)}</strong></div>`;

  if (unreadCounts[u.phone]) {
    const b = document.createElement("span");
    b.className = "unread-badge";
    b.innerText = unreadCounts[u.phone];
    d.appendChild(b);
  }

  d.onclick = () => selectRecipient(u.phone, u.username);
  l.appendChild(d);

```

```

        shownCount++;
    });

    if (shownCount === 0) {
        if (window.innerWidth <= 600 && term === "") {

            l.innerHTML = `
                <div class="empty-user-list">
                    
                    <p>End-to-End Encrypted Messaging</p>
                    <small>Search a phone number to start chatting</small></div>`;
        } else if (term === "") {
            l.innerHTML = `
                <div class="text-muted small p-3">No conversations yet.</div>`;
        } else {
            l.innerHTML = `<div class="text-muted small p-3">No registered user
found.</div>`;
        }
    }
}

```

Displays the user list, search results, chat history users, and unread message badges.

```

function clearSearch() {
    document.getElementById("user-search").value = "";
    renderUserList();
}

```

Clears the user search box and refreshes the user list.

```

function selectRecipient(p, name) {
    selectedTarget = p;
    selectedUsername = name;
    unreadCounts[p] = 0;

    document.getElementById("chat-target-name").innerText = getDisplayName(p, name);
    document.getElementById("empty-state").classList.add("hidden");
    document.getElementById("chat-header").classList.remove("hidden");
    document.getElementById("input-area").classList.remove("hidden");

    // Mobile: after selecting user, hide sidebar and show full chat
    if (window.innerWidth <= 768) {
        document.getElementById("page-chat").classList.add("mobile-chat-open");
    }
    renderUserList();
    refreshChatWindow();
}

```

Selects a receiver and opens the chat window.

```

function backToUserList() {
    if (window.innerWidth <= 768) {
        document.getElementById("page-chat").classList.remove("mobile-chat-open");
    }
}

```

```

selectedTarget = "";
selectedUsername = "";

document.getElementById("chat-target-name").innerText = "";
document.getElementById("chat-header").classList.add("hidden");
document.getElementById("input-area").classList.add("hidden");
document.getElementById("empty-state").classList.remove("hidden");
document.getElementById("message-pane").innerHTML = "";

renderUserList();
}
}

```

Returns from the chat window to the user list on mobile view.

```

function refreshChatWindow() {

  const p = document.getElementById("message-pane");
  p.innerHTML = "";

  let lastDate = "";

  (allConversations[selectedTarget] || []).forEach(m => {

    // ===== WhatsApp-style Date Divider =====
    if (m.timestamp) {
      const msgDate = new Date(m.timestamp);

      const today = new Date();

      const yesterday = new Date();
      yesterday.setDate(today.getDate() - 1);

      let dateLabel = "";

      if (msgDate.toDateString() === today.toDateString()) {
        dateLabel = "Today";
      }
      else if (msgDate.toDateString() === yesterday.toDateString()) {
        dateLabel = "Yesterday";
      }
      else {
        dateLabel = msgDate.toLocaleDateString("en-GB", {
          day: "2-digit",
          month: "long",
          year: "numeric"
        });
      }

      if (dateLabel !== lastDate) {

```

```

const divider = document.createElement("div");
divider.className = "date-divider";
divider.innerHTML = `<span>${dateLabel}</span>`;
p.appendChild(divider);

    lastDate = dateLabel;
  }
}

const d = document.createElement("div");
d.className = "msg " + (m.type === "sent" ? "msg-sent" : "msg-received");

if (m.isDeleted) {
  d.innerHTML = `<i style="opacity:0.5;font-size:13px;">🚫 This message was
deleted</i>`;
} else {
  if (m.mediaType === "audio") {
    const a = document.createElement("audio");
    a.src = m.text;
    a.controls = true;
    d.appendChild(a);
  }
  else if (m.mediaType === "image") {
    const i = document.createElement("img");
    i.src = m.text;
    i.onclick = () => i.classList.toggle("img-big");
    d.appendChild(i);
  }
  else if (m.mediaType === "video") {
    const v = document.createElement("video");
    v.src = m.text;
    v.controls = true;
    d.appendChild(v);
  }
  else {
    const text = document.createElement("span");
    text.innerText = m.text;
    d.appendChild(text);
  }
}

const meta = document.createElement("div");
meta.className = "msg-meta";

let timeText = "";
if (m.timestamp) {
  timeText = new Date(m.timestamp).toLocaleTimeString([], {
    hour: "numeric",
    minute: "2-digit"
  });
}

```

```

meta.innerHTML = timeText + (m.isEdited ? " · edited" : "");
d.appendChild(meta);

const menu = document.createElement("div");
menu.className = "msg-menu";

menu.innerHTML += `<span onclick="deleteMessage('${m.msgId}')"> 🗑️ </span>`;

if (m.type === "sent" && (!m.mediaType || m.mediaType === "text")) {
  const safeText = m.text.replace(/<\/g, "\\");
  menu.innerHTML += `<span
onclick="startEdit('${m.msgId}','${safeText}')"> ✏️ </span>`;
}

if (!m.mediaType || m.mediaType === "text") {
  const safeText = m.text.replace(/<\/g, "\\");
  menu.innerHTML += `<span onclick="copyText('${safeText}')"> 📄 </span>`;
}

d.appendChild(menu);

let pressTimer;

d.addEventListener("touchstart", function(e) {
  if (
    e.target.tagName === "AUDIO" ||
    e.target.tagName === "VIDEO" ||
    e.target.tagName === "IMG"
  ) return;

  pressTimer = setTimeout(() => {
    document
      .querySelectorAll(".msg.show-menu")
      .forEach(x => x.classList.remove("show-menu"));

    d.classList.add("show-menu");
  }, 800);
});

d.addEventListener("touchend", () => {
  clearTimeout(pressTimer);
});

d.addEventListener("touchmove", () => {
  clearTimeout(pressTimer);
});
}

p.appendChild(d);

```

```

});

p.scrollTop = p.scrollHeight;
}

document.addEventListener("click", function(e) {

  if (!e.target.closest(".msg")) {

    document
      .querySelectorAll(".msg.show-menu")
      .forEach(x =>
        x.classList.remove("show-menu")
      );
  }

});

```

Displays messages in the chat window, including text, image, video, audio, date divider, time, edit status, and message menu.

```

function togglePassword(id, btn) {
  const input = document.getElementById(id);

  if (input.type === "password") {
    input.type = "text";
    btn.innerText = "Hide";
  } else {
    input.type = "password";
    btn.innerText = "Show";
  }
}

```

Shows or hides a password input field.

```

socket.on("connect", () => {
  const phone = sessionStorage.getItem("chat_activePhone");
  if (phone) socket.emit("join", phone);
});

```

Runs when the client successfully connects to the Socket.IO server. If the user already has an active session, it automatically joins the user's chat room again.

```

socket.on("user_updated", async (data) => {
  const { oldPhone, newPhone, username } = data;

  // 1. Check if the phone number actually changed
  if (String(oldPhone) !== String(newPhone)) {
    console.log("Migration: Phone number changed by contact.");

    // If we have history for the old phone, move it to the new one
    if (allConversations[oldPhone]) {
      allConversations[newPhone] = allConversations[oldPhone];
    }
  }
});

```

```

    delete allConversations[oldPhone];

    // If we are currently chatting with them, update the target
    if (selectedTarget === oldPhone) {
        selectedTarget = newPhone;
        selectedUsername = username;
        document.getElementById("chat-target-name").innerText = username;
    }
    await saveHistory(); // Save moved data to IndexedDB
}
} else {
    // 2. Only the username changed
    console.log("Update: Contact changed their username.");
    if (selectedTarget === oldPhone) {
        selectedUsername = username;
        document.getElementById("chat-target-name").innerText = username;
    }
}

// Refresh UI to show new names
loadUsers();
if (selectedTarget === newPhone || selectedTarget === oldPhone) refreshChatWindow();
});

```

Runs when a user updates their profile. If the phone number changes, the old chat history is moved to the new phone number. If only the username changes, the chat name is updated.

```
socket.on("receive_message", d => processIncomingMessage(d));
```

Listens for incoming messages from the server and passes the message data to processIncomingMessage() for decryption and display.

```
socket.on("update_user_list", u => { window.allUsers = u; renderUserList(); });
```

Updates the frontend user list whenever the server sends a new user list.

```

socket.on("message_edited", async data => {
    const idx = allConversations[data.sender]?.findIndex(m => m.msgId === data.msgId);
    if (idx !== -1) {
        const txt = await decryptPayload(data.newPayload);
        if (txt) { allConversations[data.sender][idx].text = txt;
allConversations[data.sender][idx].isEdited = true; await saveHistory();
        if (selectedTarget === data.sender) refreshChatWindow(); }
    }
});

```

Runs when the other user edits a message. It decrypts the updated message payload and updates the local chat history.

```

socket.on("message_deleted", data => {
    const sender = data.sender;
    if (allConversations[sender]) {
        const idx = allConversations[sender].findIndex(m => m.msgId === data.msgId);
        if (idx !== -1) {

```



```

        // Change the message to the deleted marker on the recipient's side
        allConversations[sender][idx] = { msgId: data.msgId, isDeleted: true, type: "received"
    };
    saveHistory();
    if (selectedTarget === sender) refreshChatWindow();
    }
}
});

```

Runs when the other user deletes a message for everyone. It replaces the message with a deleted-message marker.

```

socket.on("user_phone_changed", async (data) => {
    const { oldPhone, newPhone, username } = data;

    // 1. If we have a chat history for the old phone, move it to the new phone
    if (allConversations[oldPhone]) {
        console.log(`System: Merging history from ${oldPhone} to ${newPhone}`);

        // Move the data in the local variable
        allConversations[newPhone] = allConversations[oldPhone];
        delete allConversations[oldPhone];

        // 2. If the user was currently looking at the old chat, switch them to the new one
        if (selectedTarget === oldPhone) {
            selectedTarget = newPhone;
            document.getElementById("chat-target-name").innerText = username;
        }

        // 3. Save the new moved history to IndexedDB
        await saveHistory();

        // 4. Refresh UI
        renderUserList();
        if (selectedTarget === newPhone) refreshChatWindow();
    }
});

```

Handles phone number migration by moving the old conversation history to the new phone number.

```

socket.on("force_logout", (data) => {
    alert(data.message || "You have been logged out because this account logged in on another device.");
    sessionStorage.clear();
    location.reload();
});

```

Forces the user to log out when the same account logs in from another device. This helps prevent multiple active sessions for one account.

```

async function handleFileSelect(event, type) {
    const files = event.target.files;

```

```

if (!files || files.length === 0) return;

const fileArray = Array.from(files);
const maxSize = 15 * 1024 * 1024; // 15MB

for (let i = 0; i < fileArray.length; i++) {
  const file = fileArray[i];

  if (file.size > maxSize) {
    alert(`File "${file.name}" is too large. Maximum file size is 15MB.`);
    continue;
  }

  await new Promise((resolve) => {
    const reader = new FileReader();

    reader.onload = async (e) => {
      await sendEncrypted(e.target.result, type);
      setTimeout(resolve, 100);
    };

    reader.readAsDataURL(file);
  });
  event.target.value = "";
}

let mediaRecorder, audioChunks = [], isRecording = false, shouldSend = true;

```

Reads selected image or video files and sends them through the encryption function.

```

async function startRecording() {
  try {
    const stream = await navigator.mediaDevices.getUserMedia({ audio: true });
    mediaRecorder = new MediaRecorder(stream); audioChunks = [];
    mediaRecorder.ondataavailable = (e) => audioChunks.push(e.data);
    mediaRecorder.onstop = async () => {
      if (shouldSend && audioChunks.length) {
        const audioBlob = new Blob(audioChunks, { type: 'audio/webm' });
        const r = new FileReader();
        r.onload = (ev) => sendEncrypted(ev.target.result, "audio");
        r.readAsDataURL(audioBlob);
      }
      stream.getTracks().forEach(t => t.stop());
    };
    mediaRecorder.start(); isRecording = true; setRecordingUI(true);
  } catch (err) { alert("Mic denied."); }
}

```

Starts microphone recording for voice messages.

```
function stopRecording() { if (mediaRecorder) { shouldSend = true; mediaRecorder.stop();
```

```
isRecording = false; setRecordingUI(false); } }
```

Stops recording and sends the audio message.

```
function cancelRecording() { if (mediaRecorder) { shouldSend = false;
mediaRecorder.stop(); isRecording = false; setRecordingUI(false); }
```

Cancels the current voice recording.

```
function toggleVoiceRecorder() { isRecording ? stopRecording() : startRecording(); }
```

Switches between starting and stopping voice recording.

```
function setRecordingUI(rec) {
  const vBtn = document.getElementById("voice-btn"); if(vBtn) vBtn.innerHTML = rec ?
  "🔴" : "🟢";
  ["cancel-record-btn", "recording-status"].forEach(id => { const el =
  document.getElementById(id); if(el) el.classList.toggle("d-none", !rec); });
  ["msg-input", "send-btn", "input-dropdown"].forEach(id => { const el =
  document.getElementById(id); if(el) el.classList.toggle("d-none", rec); });
}
```

Updates the interface when voice recording starts or stops.

```
function filterUsers() { renderUserList(); }
```

Refreshes the user list based on the search input.

```
function addKeyboardSupport() { const input = document.getElementById("msg-input");
if(input) input.addEventListener("keydown", (e) => { if (e.key === "Enter" && !e.shiftKey)
{ e.preventDefault(); sendEncrypted(); } }); }
```

Allows users to press Enter to send messages.

```
function showPage(p) { document.querySelectorAll(".container, .main-container").forEach(c
=> c.classList.add("hidden")); const el = document.getElementById(p); if(el)
el.classList.remove("hidden"); }
```

Hides all pages and displays the selected page.

```
async function loadUsers() { const r = await fetch("/list_users"); const d = await r.json();
window.allUsers = d.users; renderUserList(); }
```

Fetches all registered users from the server and displays them.

```
async function handleAuth(type) {
  // Get basic inputs
  const phone = document.getElementById(type === "login" ? "login-phone" : "reg-
phone").value.trim();
  const pass = document.getElementById(type === "login" ? "login-pass" : "reg-
pass").value;

  if (type === "signup") {
    const user = document.getElementById("reg-username").value.trim();
    const confirm = document.getElementById("reg-confirm").value;

    // 1. Check for empty fields
    if (!user || !phone || !pass || !confirm) return alert("Please fill in all registration fields.");
  }
}
```

```

// 2. Validate Username (3-20 chars, alphanumeric + underscore)
const userRegex = /^[a-zA-Z0-9_]{3,20}$/;
if (!userRegex.test(user)) {
    return alert("Username must be 3-20 characters and contain only letters, numbers, or
underscores.");
}

// 3. Validate Phone Number (10-11 digits)
const phoneRegex = /^(011\d{8}|01[0,2-9]\d{7})$/;
if (!phoneRegex.test(phone)) {
    return alert("Please enter a valid phone number.");
}

// 4. Validate Password Strength (Min 8 chars, 1 Upper, 1 Lower, 1 Number, 1 Symbol)
const passRegex = /^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@$!%*?&._-])[A-Za-
z\d@$!%*?&._-]{8,}$/;
if (!passRegex.test(pass)) {
    return alert("Security Alert: Password must be at least 8 characters and include
uppercase, lowercase, a number, and a special character.");
}

// 5. Check if passwords match
if (pass !== confirm) {
    return alert("Passwords do not match. Please re-enter.");
}

// --- VALIDATION PASSED: Generate RSA Keys ---
try {
    myKeys = await crypto.subtle.generateKey(
        {
            name: "RSA-OAEP",
            modulusLength: 2048,
            publicExponent: new Uint8Array([1, 0, 1]),
            hash: "SHA-256"
        },
        true,
        ["encrypt", "decrypt"]
    );

    const pub = await crypto.subtle.exportKey("spki", myKeys.publicKey);

    const res = await fetch("/signup", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({
            phone,
            username: user,
            password: pass,
            publicKey: arrayBufferToBase64(pub)
        })
    });

```

```

    })
  });

  const d = await res.json();
  if (!res.ok) return alert(d.message || "Registration failed.");

  myUsername = user;
} catch (err) {
  console.error("Encryption Error:", err);
  return alert("Could not generate secure keys. Please use a modern browser.");
}

} else {
  // --- LOGIN LOGIC ---
  if (!phone || !pass) return alert("Please enter your phone and password.");

  const res = await fetch("/login", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ phone, password: pass })
  });

  const d = await res.json();
  if (!res.ok) return alert(d.message || "Please check the phone number and password.");

  myUsername = d.username;
}

// Success: Set session variables and enter chat
myPhone = phone;

if (!myKeys) {
  myKeys = await crypto.subtle.generateKey(
    {
      name: "RSA-OAEP",
      modulusLength: 2048,
      publicExponent: new Uint8Array([1, 0, 1]),
      hash: "SHA-256"
    },
    true,
    ["encrypt", "decrypt"]
  );
}

await saveSession(myPhone, myKeys, myUsername);

const exportedKey = await crypto.subtle.exportKey("spki", myKeys.publicKey);
const publicKey = arrayBufferToBase64(exportedKey);

await fetch("/update_public_key", {

```

```

    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({
      phone: myPhone,
      publicKey: publicKey
    })
  });

  await checkExistingSession();
}

```

Handles both login and signup. During signup, it validates input, generates RSA keys, sends the public key to the server, and saves the session.

```

async function sendOTP() {
  const phone = document.getElementById("forgot-phone").value.trim();
  if (!phone) return alert("Enter phone number");

  const res = await fetch("/forgot_password_send_otp", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ phone })
  });

  const data = await res.json();

  if (res.ok) {
    forgotFlowData.phone = phone;

    alert("Your OTP is: " + data.otp + "\nThis OTP will expire in 5 minutes.");

    document.getElementById("forgot-step-1").classList.add("hidden");
    document.getElementById("forgot-step-2").classList.remove("hidden");
  } else {
    alert(data.message || "User not found");
  }
}

```

Requests an OTP from the server for forgot password.

```

async function verifyOTP() {
  const otp = document.getElementById("forgot-otp").value.trim();

  const res = await fetch("/forgot_password_verify_otp", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ phone: forgotFlowData.phone, otp })
  });

  const data = await res.json();

  if (res.ok) {

```

```

    forgotFlowData.otp = otp;
    document.getElementById("forgot-step-2").classList.add("hidden");
    document.getElementById("forgot-step-3").classList.remove("hidden");
  } else {
    alert(data.message || "Invalid OTP code.");
  }
}

```

Verifies the OTP entered by the user.

```

async function resetPassword() {
  const newPass = document.getElementById("forgot-new-pass").value;
  const confirmPass = document.getElementById("forgot-confirm-pass").value;

  const passwordPattern = /^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@$!%*?&._-])[A-Za-z\d@$!%*?&._-]{8,}$/;

  if (!newPass || !confirmPass) {
    return alert("Please fill in both password fields");
  }

  if (!passwordPattern.test(newPass)) {
    return alert("Password must be at least 8 characters and include uppercase, lowercase, number, and special character (@$!%*?&._-).");
  }

  if (newPass !== confirmPass) {
    return alert("Passwords do not match");
  }

  const res = await fetch("/forgot_password_reset", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({
      phone: forgotFlowData.phone,
      otp: forgotFlowData.otp,
      newPassword: newPass
    })
  });

  const data = await res.json();

  if (res.ok) {
    alert("Password updated! Please login.");
    showPage("page-login");
  } else {
    alert(data.message || "Reset failed.");
  }
}

```

Validates and submits the new password after OTP verification.

```
async function requestNotificationPermission() {  
  if ("Notification" in window) {  
    if (Notification.permission !== "granted") {  
      await Notification.requestPermission();  
    }  
  }  
}
```

Requests browser permission to show message notifications.

```
window.onload = async () => {  
  await requestNotificationPermission();  
  checkExistingSession();  
};
```

Runs when the page loads. It requests notification permission and checks for an existing session.